IS2209 Data Governance and Management

Transaction Management

Note 05

This note was created based on lecture slides and my personal understanding . Therefore, its accuracy may not be 100%. These notes reflect how I would approach the exam.

So, **use the notes at your own risk**. Good Luck!

Transaction Management

Transaction management ensures that database operations are **reliable, consistent, and safe**, even when multiple users access the database or failures occur.

It mainly covers:

1. **Transaction Processing**
2. **Concurrency Control**
3. **Recovery**

Database Transactions

A **transaction** is a sequence of one or more database operations (INSERT, UPDATE, DELETE, SELECT) treated as **one logical unit of work**.

Transactions usually reflect **real-world actions** triggered by events.

Examples

- Selling a product
- Depositing money
- Transferring money between accounts

Money Transfer Example

Account A → Balance_A -= amount

Account B → Balance_B += amount

Both updates must happen **together**, or **not at all**.

Transaction Processing

Transaction processing ensures:

- Operations are executed in the **correct order**
- All steps complete successfully before committing

Key commands:

```
START TRANSACTION;
COMMIT;
ROLLBACK;
```

Example:

```
START TRANSACTION;
UPDATE account SET balance = balance - 500 WHERE acc_no = 'A';
UPDATE account SET balance = balance + 500 WHERE acc_no = 'B';
COMMIT;
```

If any step fails → ROLLBACK.

Concurrency Control

Concurrency control manages **simultaneous transactions** accessing the same data.

Why needed?

Multiple users = risk of:

- Lost updates
- Dirty reads
- Inconsistent data

Techniques

- **Locking** (Shared / Exclusive locks)
- **Timestamp ordering**
- **Isolation levels**

Isolation Levels:

- READ UNCOMMITTED
- READ COMMITTED
- REPEATABLE READ
- SERIALIZABLE

Recovery

Recovery ensures the database can **restore itself after failures**.

Types of Failures

- System crash
- Power failure
- Disk failure
- Transaction failure

Recovery Methods

- **Undo (Rollback)** → for uncommitted transactions
- **Redo** → for committed transactions
- **Logs & Checkpoints**

Example:

- If crash happens **before COMMIT** → undo changes
- If crash happens **after COMMIT** → redo changes

Transaction Processing Systems (TPS)

A **Transaction Processing System (TPS)** is a system designed to handle a **large number of database transactions** efficiently and reliably, often with **hundreds or thousands of concurrent users**.

Examples of TPS

- Airline reservation systems
- Banking systems
- Credit card processing systems
- Online retail purchasing systems
- Stock market trading systems
- Supermarket checkout systems

Database Access Operations

Transaction processing works **independent of data item granularity** (row, column, page, etc.).

Basic Operations

- **read(X)**
  Reads the value of database item **X** into a program variable.
- **write(X)**
  Writes the value of a program variable **X** back into the database.

These two operations form the **core building blocks** of all database transactions.

Interleaved Transaction Processing

- Transactions **share CPU time**
- Operations from multiple transactions are **intermixed**
- Only **one operation executes at a time**
- Common in **single-CPU systems**

Improves CPU utilization
Needs concurrency control to avoid conflicts

Parallel Transaction Processing

- Transactions execute **simultaneously**
- Requires **multiple CPUs or cores**
- Truly concurrent execution

Faster processing
Higher throughput
More complex synchronization and locking

Quick Comparison

Feature	Interleaved	Parallel
Execution	One at a time	Simultaneous
CPU Requirement	Single CPU	Multiple CPUs
Concurrency	Logical	Physical
Complexity	Lower	Higher

What's a DB Transaction

A **transaction** is a group of database operations that must:

- **all succeed together**, or **all fail together.**

There's **no halfway state**.

If even **one step fails**, the database must be rolled back to the **previous consistent state**.

Example 1 – Balance check

```
SELECT CUST_NUMBER, CUST_BALANCE
FROM CUSTOMER
WHERE CUST_NUMBER = 1005;
```

This is a **read-only transaction**

- No data change
- Still a transaction, but very safe
- Doesn't affect consistency

What if something fails?

Let's say:

- Stock update succeeds
- Customer balance update fails

Now you'd have:

- Product stock reduced
- But customer not charged

Database becomes inconsistent

That's why this must be treated as **one atomic transaction**.

Why transactions are important

1. Data Consistency

- Database always moves from **one valid state to another**
- No broken business rules

2. Error Recovery

- If power fails, app crashes, or SQL error occurs → **ROLLBACK**
- Database goes back safely

3. Safe Concurrent Access

- Multiple users buying products at the same time
- Transactions prevent **lost updates, dirty reads, double selling**

4. Reliable System Behavior

- System behaves predictably
- Either "sale completed" or "sale failed" — nothing in between.

ACID Properties of Transactions

1. Atomicity

- A transaction is an **atomic (indivisible) unit** of processing.
- Either **all operations** in the transaction are performed, or **none** are.
- If **any operation fails**, the entire transaction is **aborted (rolled back)**.

All or nothing.

2. Consistency Preservation

- A transaction must take the database from **one consistent state to another**.
- All **integrity constraints** (keys, rules, validations) must be satisfied.
- If executed fully **without interference**, consistency is guaranteed.

No rule breaking.

3. Isolation

- Each transaction must appear to execute **independently** of others.
- Intermediate results of a transaction **must not be visible** to other transactions.
- Prevents issues like:
    - Dirty reads
    - Lost updates
    - Inconsistent reads
- Especially important in **multi-user environments**.

Transactions don't interfere.

4. Durability (Permanency)

- Once a transaction is **committed**, its changes are **permanent**.
- Changes must survive:
    - System crashes
    - Power failures
- Achieved using logs, backups, and recovery mechanisms.

Committed = forever.

Transaction Management with SQL

SQL provides two main statements:

COMMIT

- Saves all changes permanently.
- Ends the transaction.

ROLLBACK

- Cancels all changes.
- Restores the database to the **previous consistent state**.
- Ends the transaction.

How a Transaction Ends

A transaction continues until **one** of these occurs:

1. **COMMIT**
    - Changes are permanently stored.
2. **ROLLBACK**
    - Changes are undone.
3. **Program ends normally**
    - Equivalent to COMMIT.
4. **Program terminates abnormally**
    - Equivalent to ROLLBACK.

Example Transaction

```sql
BEGIN TRANSACTION;

UPDATE PRODUCT
SET PROD_QOH = PROD_QOH - 2
WHERE PROD_CODE = '1558-QW1';

UPDATE CUSTOMER
SET CUST_BALANCE = CUST_BALANCE + 87.98
WHERE CUST_NUMBER = '10011';

COMMIT;
```

What this ensures:

- **Atomicity** → Both updates succeed or both fail
- **Consistency** → Stock and balance remain valid
- **Isolation** → Other users don't see partial updates
- **Durability** → Changes persist after commit

Transaction Log (Log Files)

A **transaction log** is a **sequential file** that records **all database changes** made by transactions. It is mainly used for **recovery** in case of failures (system crash, power failure, etc.).

Why Log Files Are Needed

To recover the database correctly, the system must know:

- **BFIM (Before Image)** → value *before* modification
- **AFIM (After Image)** → value *after* modification

These allow the DBMS to:

- **UNDO** incomplete transactions (using BFIM)
- **REDO** committed transactions (using AFIM)

What Is Stored in a Log Record

Each log record typically contains:

Field	Meaning
TID	Transaction ID
Back P	Pointer to previous log record of the same transaction
Next P	Pointer to next log record of the same transaction
Operation	Begin / Read / Write / End
Data Item	Data item accessed
BFIM	Value before update
AFIM	Value after update

Back P and Next P help trace all operations of a specific transaction.

Sample Transaction Log (Simplified Explanation)

TID	Back P	Next P	Operation	Data Item	BFIM	AFIM
T1	0	1	Begin	—	—	—
T1	1	4	Write	X	100	200
T2	0	8	Begin	—	—	—
T1	4	7	Read	M	200	200
T3	0	9	Read	N	400	400
T2	2	5	Write	Y	50	100
T1	5	NIL	End	—	—	—

How Recovery Uses the Log

If a transaction committed

➞ **REDO** using **AFIM**

If a transaction did not commit

➞ **UNDO** using **BFIM**

Key Points for Exams

- Transaction log is **essential for recovery**
- Stores **BFIM and AFIM**
- Log records are **written sequentially**
- Enables **UNDO and REDO operations**
- Back and Next pointers help trace a transaction

Concurrent Execution of Transactions

When multiple transactions execute **simultaneously** in a shared multi-user database, **concurrency problems** can occur.

Common Problems in Concurrent Execution

- **Lost Update Problem**
- **Temporary Update (Dirty Read) Problem**
- **Incorrect Summary Problem**
- **Unrepeatable Read Problem**

These issues arise when transactions **interfere with each other** while accessing the same data items.

Lost Update Problem

A **lost update** occurs when two transactions access the same data item concurrently, and one transaction's update **overwrites** the other's update because the operations are not properly isolated.

Example scenario:

- Initial values: X = 80, Y = 100, N = 5, M = 4
- Transaction T1 wants to subtract N from X and add N to Y.
- Transaction T2 wants to add M to X.
- Because the transactions run concurrently, some updates may be **lost**, resulting in an inconsistent database state.

Interleaved Transactions Table

T1	**T2**
READ(X) → X = 80	
X = X - N → X = 75	
	READ(X) → X = 80
	X = X + M → X = 84
WRITE(X) → X = 75	
READ(Y) → Y = 100	
	WRITE(X) → X = 84
Y = Y + N → Y = 105	
WRITE(Y) → Y = 105	

Step-by-Step Explanation

1. **T1 READ(X)**: T1 reads X = 80.
2. **T1 calculates X**: T1 subtracts N → X = 75.
3. **T2 READ(X)**: T2 reads X = 80 (before T1 writes).
4. **T2 calculates X**: T2 adds M → X = 84.
5. **T1 WRITE(X)**: T1 writes X = 75 to the database. T2's pending update (X = 84) is **overwritten later**, causing T1's update to be **lost**.
6. **T1 READ(Y)**: T1 reads Y = 100.
7. **T2 WRITE(X)**: T2 writes X = 84. T1's previous write (75) is now **lost**.
8. **T1 calculates Y**: T1 adds N → Y = 105.
9. **T1 WRITE(Y)**: T1 writes the new value Y = 105.

Result: The update by T1 to X is **lost**, demonstrating the lost update problem.

Temporary Update Problem (Uncommitted Data / Dirty Read)

A **temporary update problem** occurs when a transaction updates a data item but **fails or rolls back** before committing. If another transaction reads the updated value **before it's rolled back**, it reads **"dirty" data**, which may never actually exist in the database.

Example scenario:

- Initial values: X = 80, Y = 100, N = 5, M = 4
- Transaction T1 subtracts N from X but then **rolls back**.
- Transaction T2 reads X while T1's update is still uncommitted.

Interleaved Transactions Table

T1	**T2**
READ(X) → X = 80	
X = X - N → X = 75	
WRITE(X) → X = 75	
	READ(X) → X = 75
	X = X + M → X = 79
	WRITE(X) → X = 79
READ(Y) → Y = 100	
ROLLBACK	

Step-by-Step Explanation

1. **T1 READ(X)**: T1 reads X = 80.
2. **T1 calculates X**: X = X - N → X = 75.
3. **T1 WRITE(X)**: X = 75 is written to the database **but not committed yet**.
4. **T2 READ(X)**: T2 reads X = 75 — this is **dirty/uncommitted data** because T1 may still roll back.
5. **T2 calculates X**: X = X + M → X = 79.
6. **T2 WRITE(X)**: X = 79 is written to the database.
7. **T1 READ(Y)**: T1 reads Y = 100.
8. **T1 ROLLBACK**: All changes by T1 are undone, including X = 75.

Result: T2 used **uncommitted data** from T1 to calculate X, so the final value (X = 79) may not reflect the **true consistent state** of the database. This is why reading uncommitted data is dangerous.

Incorrect Summary Problem

Happens when a transaction calculates an **aggregate function** (like sum) while other transactions update some of the items. The aggregate may read some values **before** updates and some **after**, producing a **wrong result**.

Example Values:

- X = 80, Y = 100, N = 5

Interleaved Schedule Table (T1 | T3)

T1	**T3**
	Sum = 0
	READ(A)
	Sum += A
READ(X)	
X = X - N	
WRITE(X)	
	READ(X)
	Sum += X
	READ(Y)
	Sum += Y
READ(Y)	
Y = Y + N	
WRITE(Y)	

Explanation:

- T3 starts calculating sum first, but reads **X after T1 updated it** and **Y before T1 updates it**, giving a **wrong aggregate**.
- Correct sum should be A + (X initial - N) + (Y initial + N), but interleaving breaks this.

Solutions:

1. Use **serializable isolation** to prevent interleaving.
2. Lock data items during aggregation.
3. Use **atomic aggregate operations** if supported.

Dirty Data / Dirty Read

A **dirty read** happens when a transaction reads data that has been **written by another transaction that has not yet committed**.

- If the first transaction later **rolls back**, the reading transaction has read invalid or "dirty" data.

Example 1 (Employee Salary):

Transaction T1: Update Employee set Salary = Salary + 1000 where eid=123;

Transaction T2: Select Avg(Salary) from Employee;

- If T2 reads **before T1 commits**, it may calculate the average including the **uncommitted salary change**.
- If T1 later rolls back, T2 has read **dirty data**.

Key: A row is considered dirty if it has been **inserted, updated, or deleted by another uncommitted transaction**.

Example 2 (Students)

Transactions:

1. T1: Update Students set GPA = 1.1 * GPA where marks > 90;
2. T2: Select GPA from Student where sid=123;
3. T1: Update Student set marks = 94 where sid=234;

Dirty Read?

- If **T2 reads the GPA while T1 has updated it but not committed**, T2 has performed a **dirty read**.
- If T1 later **rolls back**, T2's read was invalid.

Key Points:

- Dirty reads **only occur across transactions**.
- Happens due to **lack of proper isolation**.
- Prevented by **higher isolation levels** like **Read Committed** or **Serializable**.

Unrepeatable Read Problem

Happens when a transaction reads the same data item **twice**, but another transaction **modifies that item in between**.

- As a result, the first transaction sees **different values** for the same item, leading to inconsistent reads.

Example:

- Initial value: X = 80

Transactions:

- **T1:** Reads X twice
- **T2:** Updates X by subtracting 5

Interleaved Schedule (T1 | T2)

T1	**T2**
READ(X)	
	READ(X)
	X = X - 5
	WRITE(X)
READ(X)	

Explanation:

1. T1 reads X the first time → sees 80.
2. T2 reads X, subtracts 5, writes X → X becomes 75.
3. T1 reads X the second time → sees 75, **different from the first read**.

Impact:

- Leads to **inconsistent results** in transactions relying on repeated reads.
- Can cause errors in calculations, reports, or decision-making if the transaction expects the value to remain the same.

Solution:

- Use **Repeatable Read** or **Serializable isolation levels** to prevent other transactions from modifying the data between reads.

Phantom Problem

A **phantom** is a row that **matches a search condition** but **was not visible** to a transaction initially.

- Occurs when **another transaction inserts or updates rows** that satisfy the search condition of the first transaction.
- When the first transaction re-executes the query, it sees **additional (phantom) rows**.

Example:

- **T1:** Reads all students with marks > 90.
- **T2:** Inserts a new student with marks = 95.
- **T1:** Reads the query again → now sees the new student row (**phantom**).

Interleaved Schedule (T1 | T2)

T1	**T2**
READ Students where marks > 90	
	INSERT new student marks=95
READ Students where marks > 90	

Explanation:

1. T1 reads a set of rows satisfying its condition → sees the initial set.
2. T2 inserts or updates a row that also satisfies the condition.
3. T1 re-executes the query → now the new row appears (**phantom row**).

Impact:

- Leads to **inconsistent query results** for repeated executions within the same transaction.
- Can affect reporting, aggregates, or decision-making if consistency is expected.

Solution:

- Use **Serializable isolation level** to prevent phantoms.
- Can also use **range locks** to block inserts/updates in the query range.

Concurrency Control Techniques

1. Locking

- **Purpose:** Prevent conflicting access to data by multiple transactions.
- **Types:**
  - **Shared Lock (S):** Multiple transactions can **read** the same data simultaneously.
    - **Example:**

```
T1: READ(X)  → acquires shared lock
T2: READ(X)  → acquires shared lock
```

  - **Exclusive Lock (X):** Only one transaction can **write** to the data at a time.
    - **Example:**

```
T1: WRITE(X) → acquires exclusive lock
T2: WRITE(X) → must wait until T1 releases lock
```

2. Timestamp Ordering

- **Purpose:** Orders transactions based on **start time** rather than locks.
- **How it works:**
  - Each transaction gets a **timestamp**.
  - Reads/writes are allowed only if they respect timestamp order; otherwise, the transaction may **abort and restart**.
- **Example:**

```
T1 (timestamp 1) wants to write X
T2 (timestamp 2) wants to read X
→ T2 can only read the previous committed value, otherwise it rolls back
```

Isolation Levels

Isolation levels determine **how visible the changes of one transaction are to other concurrent transactions**. They control the degree of interference between transactions and prevent concurrency problems like **dirty reads, unrepeatable reads, and phantom reads**.

1. READ UNCOMMITTED

- **Description:**
  - Lowest isolation level.
  - Transactions can **read uncommitted changes** from other transactions.
- **Problems Allowed:**
  - **Dirty reads** (reading uncommitted data)
  - **Unrepeatable reads**
  - **Phantoms**
- **Example:**

```
T1: UPDATE X = 100 (not committed)
T2: READ X → sees 100 (dirty read)
T1: ROLLBACK → X goes back to 80, T2 read invalid data
```

2. READ COMMITTED

- **Description:**
  - Transactions can **only read committed data**.
  - Prevents dirty reads.
- **Problems Allowed:**
  - **Unrepeatable reads**
  - **Phantoms**
- **Example:**

```
T1: UPDATE X = 100 (not committed)
T2: READ X → sees old value 80
T1: COMMIT → X = 100
T2: READ X again → sees 100 (unrepeatable read)
```

3. REPEATABLE READ

- **Description:**
  - Ensures that **if a transaction reads a row twice, it sees the same value both times**.
  - Prevents **dirty reads** and **unrepeatable reads**.
- **Problems Allowed:**
  - **Phantoms** (new rows can still appear)
- **Example:**

```
T1: READ all students with marks > 90
T2: UPDATE X in that set → blocked until T1 finishes
T1 reads again → sees same values for X
T2: INSERT new student → allowed → phantom
```

4. SERIALIZABLE

- **Description:**
  - Highest isolation level.
  - Ensures **full isolation** as if transactions executed one after another.
  - Prevents **dirty reads, unrepeatable reads, and phantoms**.
- **Example:**

```
T1: READ all students with marks > 90
T2: INSERT student with marks > 90 → blocked until T1 finishes
T1 reads again → sees same set, no phantoms
```

Summary Table:

Isolation Level	Dirty Read	Unrepeatable Read	Phantom
READ UNCOMMITTED	Yes	Yes	Yes
READ COMMITTED	No	Yes	Yes
REPEATABLE READ	No	No	Yes
SERIALIZABLE	No	No	No